

Universidade Estadual de Mato Grosso do Sul

Curso de Ciência da Computação

Disciplina de Redes de Computadores

Trabalho de Redes de Computadores

**PLATAFORMA DE MONITORAMENTO INTELIGENTE DE FILAS
EM TEMPO REAL**

Documentação

Victor Rech Vendruscolo

Dourados (MS), 29 de julho de 2026.

Sumário

1	Sumário do problema	1
2	Algoritmos, tipos de dados, funções e decisões	1
2.1	Arquitetura geral	1
2.2	Tipo abstrato de dados: a fila	2
2.3	Protocolo de mensagens	3
2.4	Módulos e funções	4
2.5	Algoritmos principais	4
2.6	Decisões de implementação	5
2.7	Duas construções que merecem explicação	6
3	Decisões de implementação omissas na especificação	7
3.1	Elementos acrescentados por necessidade do protocolo	7
3.2	Pontos ambíguos e como foram resolvidos	7
3.3	Escopo declarado: o que não foi implementado	8
3.4	Limitações conhecidas	8
3.5	Persistência em arquivos de texto	9
4	Retransmissão de mensagens	9
4.1	O que precisava ser garantido	9
4.2	Mecanismo adotado	10
4.3	Avaliação honesta do mecanismo	10
5	Testes e análise	10
5.1	Como os testes foram feitos	10
5.2	Teste funcional	11
5.3	Casos especiais: 30 de 30 aprovados	11
5.4	Retransmissão vista do lado do cliente	13
5.5	Compilação	13
6	Teste de carga com 100, 1.000 e 10.000 clientes	13
6.1	Como a carga foi gerada	13
6.2	Resultados	14
6.3	Análise dos resultados	15
7	Telas do funcionamento correto	16
8	Conclusão	18
	Referências bibliográficas	18

1 Sumário do problema

Este trabalho implementa uma plataforma cliente-servidor para monitorar uma fila de atendimento em tempo real. Um servidor central mantém a fila; vários clientes se conectam ao mesmo tempo e operam sobre ela. O sistema foi escrito em C, sobre sockets TCP, e roda em Linux.

O problema tratado é a falta de informação compartilhada em locais de atendimento ao público. Quando cada posto anota a fila por conta própria, ninguém sabe o estado real do atendimento. O enunciado cita superlotação, filas desorganizadas e ausência de atualização em tempo real como sintomas disso.

A solução adotada centraliza a fila em um único servidor. Todo cliente que cadastra um usuário altera a mesma estrutura, e todos os demais são avisados na hora. Assim, o estado da fila é único e visível para todos os operadores conectados.

O sistema oferece quatro operações, que reproduzem as telas do enunciado:

- **Autenticação:** o cliente se identifica ao conectar, com uma credencial fixa definida no código.
- **Adicionar usuário:** o operador informa um identificador e um nome, que entram na fila compartilhada.
- **Ver fila:** o servidor devolve a fila completa, do primeiro ao último registro.
- **Heartbeat:** o servidor devolve apenas os usuários cadastrados por outros clientes desde a última verificação. Se não houver novidade, devolve ALIVE.

Além dessas quatro operações pedidas, o servidor envia uma notificação espontânea. Sempre que alguém cadastra um usuário, todos os outros clientes recebem a mensagem de *broadcast* sem terem pedido nada. É esse mecanismo que caracteriza o tempo real do sistema.

Terminologia. As telas do enunciado usam a palavra “paciente”, mas o texto do enunciado usa “usuário”. Este documento adota **usuário** para quem entra na fila e **cliente** para o programa ./cliente que se conecta e opera o sistema. São dois conceitos diferentes e a distinção vale para todo o documento.

2 Algoritmos, tipos de dados, funções e decisões

2.1 Arquitetura geral

O sistema tem dois programas, ligados por uma conexão TCP na porta 8080. A persistência fica em arquivos de texto, do lado do servidor.

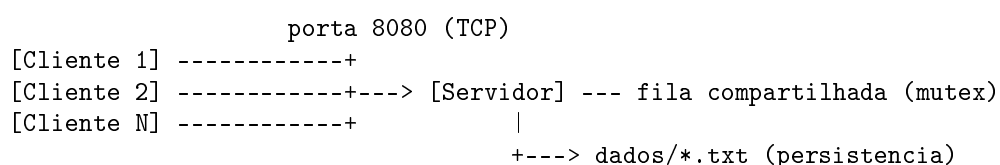


Figura 1: Arquitetura do sistema, correspondente ao diagrama do enunciado.

A técnica de múltiplos clientes escolhida foi **multithreads** (pthreads), entre as três permitidas pelo enunciado: multiplexação, `fork()` e `threads`. O servidor cria uma thread para cada conexão aceita.

O que decidiu a escolha foi a fila compartilhada. Com `threads`, todas enxergam a mesma memória, e basta um *mutex* para proteger o acesso; o *broadcast* também alcança os sockets das outras conexões sem nenhum mecanismo extra. Com `fork()`, as duas coisas exigiriam comunicação entre processos, porque cada processo tem memória isolada. A multiplexação com `select()` resolveria o compartilhamento igualmente bem, e sem precisar de *mutex*; ficou de fora por ser a técnica menos trabalhada ao longo do curso, e o prazo não permitiria aprendê-la a ponto de explicá-la com segurança.

A thread principal do servidor não atende nenhum cliente. Ela apenas aceita conexões, cria a thread correspondente e volta a aceitar. Isso atende ao enunciado, que reserva ao servidor principal apenas aceitar conexões, distribuir eventos e monitorar sockets.

```
for (;;) {
    novo_socket = accept(socket_escuta, ...);
    ...
    dados = (DadosConexao *) malloc(sizeof(DadosConexao));
    dados->sock = novo_socket;
    dados->porta = ntohs(endereco_cliente.sin_port);
    inet_ntop(AF_INET, &endereco_cliente.sin_addr, dados->ip,
        ...);
    dados->numero = ++total_conexoes;
    pthread_create(&identificador_thread, &atributos,
        atende_cliente, dados);
}
```

Código 1: Laço principal do servidor, em `servidor.c`. A thread criada recebe uma struct própria, com o socket, o IP e a porta de origem e o número da conexão.

2.2 Tipo abstrato de dados: a fila

A fila é o dado central do sistema. Ela é um vetor de tamanho fixo, protegido por um único *mutex*, e todo acesso passa pelas funções do módulo `fila.c`. Manter o acesso concentrado em um módulo torna impossível esquecer de travar.

```
typedef struct {
    int id; /* identificador digitado pelo
operador */
    char nome[TAM_NOME]; /* nome, terminado em '\0'
*/
} Usuario;

static Usuario fila[MAX_USUARIOS_FILA]; /* 20000
registros */
static int quantidade; /* posicoes em
uso */
static pthread_mutex_t fila_mutex; /* protege os
dois */
```

Código 2: Tipo `Usuario` e estado interno da fila.

O vetor foi preferido à lista ligada pela simplicidade. O limite de 20.000 registros

cobre com folga o teste de carga de 10.000 clientes. Esse limite é uma decisão de implementação declarada na Seção 3.

Cada conexão guarda, por conta própria, um marcador do último índice da fila que já viu. É esse marcador que permite ao *heartbeat* devolver apenas o que é novo para aquele cliente.

2.3 Protocolo de mensagens

Toda mensagem é uma linha de texto ASCII terminada em `\n`. O comando vem em maiúsculas, seguido dos argumentos separados por espaço. Essa escolha resolve o problema central de trabalhar sobre TCP: o TCP entrega um fluxo de bytes, não mensagens delimitadas, e uma chamada de `send()` não corresponde necessariamente a uma de `recv()`. Quem lê consome um byte por vez até encontrar a quebra de linha, e devolve à camada de cima exatamente uma mensagem por chamada.

Respostas de várias linhas usam um cabeçalho e um rodapé fixos. O cliente lê linha a linha até bater no rodapé, sem precisar saber quantas linhas virão.

Tabela 1: Mensagens do cliente para o servidor.

Operação	Formato	Quando
Login	LOGIN <usuario> <senha>	Primeira mensagem da sessão
Adicionar usuário	ADD <seq> <id> <nome>	Opção 1 do menu
Ver fila	LIST <seq>	Opção 2 do menu
<i>Heartbeat</i>	HEARTBEAT <seq>	Opção 3 do menu
Sair	SAIR	Opção 0 do menu

Tabela 2: Mensagens do servidor para o cliente.

Resposta	Formato	Quando
Login aceito	LOGIN_OK	A credencial confere
Login recusado	LOGIN_FAIL	A credencial não confere; o servidor fecha a conexão
Usuário inserido	ADD_OK <id> <nome>	Após gravar na fila e disparar o <i>broadcast</i>
Fila	==== FILA ===== <id> - <nome> × N =====	Resposta de LIST
<i>Heartbeat</i> com novidade	Mesmo formato da fila, só com os registros de outros clientes	Resposta de HEARTBEAT
<i>Heartbeat</i> sem novidade	ALIVE	Resposta de HEARTBEAT
Erro	ERRO <descrição>	Comando inválido; a conexão continua aberta
<i>Broadcast</i>	[Broadcast] Novo usuario: <nome>	Mensagem não solicitada, enviada a todos os clientes menos ao autor do ADD

O número de sequência <seq> acompanha todo comando posterior ao login. Ele começa em 1 e aumenta a cada novo comando daquela conexão. Sua função é sustentar a retransmissão, tratada na Seção 4.

2.4 Módulos e funções

O código está dividido em seis módulos, além dos dois programas principais e do gerador de carga. Cada arquivo tem uma responsabilidade só.

Tabela 3: Módulos do sistema e suas funções principais.

Arquivo	Responsabilidade	Funções principais
comum.h	Constantes, tipo <code>Usuario</code> e textos do protocolo	Somente definições
protocolo.c/.h	Envio e leitura de mensagens delimitadas por linha	protocolo_le_linha, protocolo_envia_linha, protocolo_limpa_bordas
fila.c/.h	Fila compartilhada protegida por <i>mutex</i>	fila_init, fila_adiciona, fila_tamanho, fila_copia_intervalo
sessoes.c/.h	Registro dos clientes conectados e envio do <i>broadcast</i>	sessoes_init, sessoes_registra, sessoes_remove, sessoes_broadcast, sessoes_trava_envio, sessoes_libera_envio
persistencia.c/.h	Gravação nos quatro arquivos de texto	persistencia_init, persistencia_log_servidor, persistencia_log_sessao, persistencia_historico_add, persistencia_salva_fila
servidor.c	Aceitação de conexões e atendimento	main, atende_cliente, autentica, trata_add, trata_list, trata_heartbeat, envia_bloco_fila, envia_resposta, verifica_sequencia, guarda_resposta_add, nome_valido
cliente.c	Interface de operação	main, conecta_ao_servidor, autentica, thread_receptora, envia_comando, imprime_resposta, opcao_adicionar_usuario, opcao_ver_fila, opcao_heartbeat, mostra_menu
carga.c	Gerador automático de clientes para o teste de carga	main, conecta, autentica
Makefile	Compilação	Alvos all (padrão), carga, tudo, clean

O `Makefile` sem parâmetros gera exatamente os dois programas exigidos, `cliente` e `servidor`. O gerador de carga ficou em um alvo separado, `make carga`, justamente para não acrescentar um terceiro executável ao alvo padrão.

Não existem `servidor.h` nem `cliente.h`. Um cabeçalho não faz sentido para um módulo que só contém o `main` e funções privadas ao arquivo, todas declaradas `static`.

2.5 Algoritmos principais

Atendimento de um cliente

Cada thread executa sempre a mesma sequência. Ela autentica a conexão, registra a sessão, processa comandos em laço e limpa tudo ao final.

1. Ler a primeira linha e conferir a credencial. Se falhar, responder LOGIN_FAIL e fechar a conexão.
2. Registrar a sessão na tabela de sessões, para o *broadcast* encontrá-la.
3. Em laço: ler uma linha, identificar o comando pela primeira palavra e chamar o tratador correspondente.
4. Ao receber SAIR, ou quando o `recv()` indicar que a conexão caiu, remover a sessão e fechar o socket.

Inserção e *broadcast*

O tratador de ADD valida os dados, insere na fila e avisa os demais. A ordem importa: a inserção acontece antes da notificação, para nenhum cliente ser avisado de um registro que ainda não existe.

1. Separar sequência, identificador e nome da linha recebida.
2. Validar: identificador positivo, nome não vazio e dentro do limite de 49 caracteres.
3. Conferir o número de sequência. Se for repetido, devolver a resposta guardada e encerrar aqui.
4. Inserir na fila, com o *mutex* travado.
5. Gravar no histórico e reescrever o retrato da fila.
6. Enviar o *broadcast* a todas as sessões, exceto à de origem.
7. Responder ADD_OK ao autor e guardar essa resposta.

Heartbeat

O algoritmo compara o marcador da conexão com o tamanho atual da fila. Se o tamanho for maior, o servidor devolve os registros do intervalo e avança o marcador. Se não, devolve ALIVE.

Há um detalhe necessário para a regra funcionar. Quando o próprio cliente faz um ADD, seu marcador avança na hora. Sem isso, o *heartbeat* devolveria a ele o usuário que ele mesmo acabou de cadastrar, o que contraria a definição de “usuários cadastrados por outros clientes”.

2.6 Decisões de implementação

Tabela 4: Principais decisões tomadas durante a codificação.

Decisão	Motivo
O cliente também usa duas threads: uma lê o socket, a outra lê o teclado	Com um fluxo só, o programa ficaria preso no <code>fgets</code> e a notificação de <i>broadcast</i> só apareceria quando o operador digitasse algo. Isso contraria a exigência de atualização em tempo real. A thread receptora é a única que chama <code>recv()</code>
O texto que trafega na rede é ASCII puro; a acentuação existe só na exibição	O enunciado especifica texto ASCII e o tráfego será comparado com esta documentação. Bytes iguais em qualquer máquina evitam divergência por codificação. O cliente reescreve a mensagem com acento apenas ao mostrá-la na tela
A fila é lida em lotes de 32 registros ao ser enviada	Mantém o <i>mutex</i> travado por intervalos curtos. Uma listagem longa não impede outros clientes de inserir
O nome é validado uma única vez, no servidor	Corrige um defeito encontrado em teste. O nome era cortado ao entrar na fila, mas a confirmação e o histórico usavam o texto original. Hoje fila, histórico, <i>broadcast</i> e resposta usam exatamente o mesmo texto
O socket vai para a thread dentro de uma <i>struct</i> alocada, não como inteiro convertido em ponteiro	Converter <code>int</code> em <code>void*</code> não é portátil em 64 bits. A <i>struct</i> ainda permite levar o IP de origem e o número da conexão junto
As threads são criadas já desatachadas	Equivale a chamar <code>pthread_detach</code> logo após a criação, com uma chamada a menos por conexão. Faz diferença em 10.000 conexões
O servidor não tem encerramento ordenado	Ele roda até ser interrompido. Como toda gravação é seguida de <code>fflush</code> , nada se perde, e o sistema operacional libera os recursos do processo. Evita código que nunca é executado

2.7 Duas construções que merecem explicação

O código usa dois recursos que não aparecem nos exemplos vistos em aula. Como são os pontos que menos se reconhecem numa primeira leitura, ficam explicados aqui.

A espera com prazo, no cliente. A thread do menu envia um comando e precisa aguardar a resposta — mas quem recebe a resposta é a outra thread, a que lê o socket. As duas se comunicam por uma variável de condição (`pthread_cond_timedwait`): a thread do menu dorme até ser avisada, ou até o prazo de 3 segundos terminar. O prazo é informado como um instante absoluto, `time(NULL)` mais 3, porque é isso que a função espera receber. A espera fica dentro de um `while`, e não de um `if`, porque uma variável de condição pode acordar sem ninguém ter sinalizado; a condição é reconferida a cada acorde.

A separação do nome, no servidor. O comando de cadastro chega como uma linha só, e o nome pode conter espaços — então não dá para separar a linha em palavras. A leitura é feita com `sscanf(linha, "%s %d %d %n", ...)`. O `%s` lê a primeira palavra e a descarta, porque o comando já é conhecido. Os dois `%d` leem o número de sequência e o identificador. O `%n` não lê nada: ele grava quantos caracteres foram consumidos até ali, o que marca exatamente onde o nome começa. O resto da linha, a partir desse ponto, é o nome inteiro.

3 Decisões de implementação omissas na especificação

O enunciado exige que qualquer elemento não especificado, mas necessário para o protocolo funcionar, seja descrito de forma explícita. Esta seção é essa declaração. Ela cobre quatro grupos: o que foi acrescentado, o que foi decidido diante de ambiguidade, o que ficou fora de escopo e as limitações conhecidas.

3.1 Elementos acrescentados por necessidade do protocolo

1. **Tabela de sessões** (`sessoes.c`). O *broadcast* exige saber, a cada instante, quais sockets estão conectados e autenticados. Implementada como vetor de tamanho fixo (`MAX_SESSOES = 12000`), protegido por um *mutex*.
2. **Serialização de escrita**. Sem ela, uma mensagem de *broadcast* disparada por outra thread poderia entrar no meio de uma resposta de várias linhas e quebrar o protocolo. Existe um único *mutex* de envio, comum a todos os sockets, mantido travado do cabeçalho ao rodapé do bloco. Os três *mutex* do servidor são sempre obtidos na mesma ordem — tabela de sessões, envio, fila —, e nenhum caminho do código inverte essa ordem. É isso que descarta a possibilidade de impasse (*deadlock*).
3. **Sinal SIGPIPE ignorado**, no cliente e no servidor. Escrever em um socket que o outro lado já fechou gera esse sinal, cujo comportamento padrão é encerrar o processo inteiro. Ignorá-lo transforma a situação em um erro de retorno, que o código já trata.
4. **Pilha reduzida das threads**, de 8 MB para 256 KB, definida por `pthread_attr_setstacksize`. Com o valor padrão, 10.000 threads reservariam 80 GB de espaço de endereçamento e o teste de carga seria impossível.
5. **Numeração das conexões nas linhas de log**. As mensagens começam exatamente com o texto das telas do enunciado e recebem, no fim, o número sequencial e a origem — por exemplo, Novo cliente conectado. [4271] 192.168.0.20:51344. Sem isso, o print do teste de 10.000 clientes seria uma parede de linhas idênticas, e o enunciado exige que seja possível identificar todas as conexões.

3.2 Pontos ambíguos e como foram resolvidos

Tabela 5: Pontos em aberto no enunciado e a decisão adotada.

Ponto	Decisão
<i>Heartbeat</i> : texto da Observação contra o <i>print</i>	A Observação diz que a função devolve os usuários cadastrados por outros clientes, ou ALIVE. O <i>print</i> mostra a fila inteira. Seguimos o texto, por ser a especificação escrita
“Atualizar filas em tempo real” no cliente	Implementado como notificação assíncrona do novo usuário, não como réplica local da fila. É o que os <i>prints</i> mostram. O operador pede a fila completa com a opção 2 quando quiser
Terminologia “paciente” contra “usuário”	Adotado “usuário”, termo usado no texto do enunciado
Persistência: banco de dados ou arquivo	Arquivo de texto, opção permitida pelo enunciado. Quatro arquivos, descritos na Seção 3.5

Ponto	Decisão
Endereço do servidor	Constante <code>SERVER_IP</code> em <code>comun.h</code> , alterada e recompilada ao mudar de máquina. Nenhum código visto na disciplina faz descoberta dinâmica de rede
Autenticação	Usuário e senha fixos no código, sem validação contra arquivo ou banco. Uma credencial única, coerente com o sistema ter um único tipo de operador
Unicidade de identificador	Não há verificação. Dois usuários podem ter o mesmo identificador
Capacidade da fila	Limite fixo de 20.000 registros. Ao encher, o servidor responde <code>ERR0 Fila cheia</code> sem derrubar nada
Tamanho do nome	Até 49 caracteres. Acima disso o cadastro é recusado, não cortado
Identificador inválido	Recusado se for menor ou igual a zero
Recuperação da fila	O servidor sempre inicia com a fila vazia e não recarrega <code>fila.txt</code> . Os arquivos de log e histórico continuam existindo entre execuções

3.3 Escopo declarado: o que não foi implementado

A introdução do enunciado cita algumas funcionalidades. Algumas ficaram fora de escopo, e a decisão está declarada aqui, como o enunciado exige.

Tabela 6: Funcionalidades citadas apenas na introdução e não implementadas.

Item	Motivo
Painéis administrativos	Citado apenas na introdução. Exigiria um tipo de cliente com interface própria, que não aparece em nenhuma tela
Balanceamento de carga	Pressupõe mais de um servidor. A arquitetura do enunciado mostra um servidor central único
Métricas em tempo real para administradores	Depende do painel administrativo
Papéis distintos de recepcionista, coordenador e administrador	O sistema tem um único tipo de operador, coerente com a credencial única prevista para o login
Posição numérica do usuário na fila	A resposta de <code>LIST</code> segue o formato exato do <i>print</i> (<code>id - nome</code>), sem número de posição. A ordem das linhas já é a ordem de atendimento
Acompanhamento via aplicativo ou página web	O enunciado determina cliente e servidor em C, iniciados por linha de comando

3.4 Limitações conhecidas

Três comportamentos foram identificados durante os testes e ficam declarados aqui, em vez de contornados.

Estouro de inteiro no identificador. O identificador é lido com `sscanf(entrada, "%d",`

&id). Um número acima do limite do tipo `int` sofre estouro silencioso e é gravado com outro valor: em teste, 9999999999999999 entrou na fila como 276447231. Negativos e zero são recusados, mas o estouro produz um valor positivo, então passa pela validação. Corrigir exigiria voltar a `strtol` com verificação de `errno`. É um comportamento comum em C e sem efeito sobre o funcionamento do sistema.

O teto de clientes é menor que o da fila. A fila cabe 20.000 usuários, mas a tabela de sessões cabe 12.000 (`MAX_SESSOES`). Portanto o número máximo de clientes conectados ao mesmo tempo é 12.000, e não 20.000. Os dois limites são independentes: um cliente pode cadastrar vários usuários. O valor de 12.000 cobre com folga o maior teste exigido, que é de 10.000 clientes.

O retrato da fila é reescrito por inteiro. A cada inserção, o arquivo `fila.txt` é regravado do começo, e não acrescido de uma linha. O custo cresce com o tamanho da fila. A escolha foi essa porque o arquivo precisa refletir o estado atual, no mesmo formato da resposta de `LIST`; o registro que só cresce é o `historico.txt`, que recebe uma linha por inserção. Com o volume deste trabalho a diferença não aparece.

3.5 Persistência em arquivos de texto

O enunciado permite escolher entre banco de dados e arquivo. A escolha foi arquivo de texto, e os dados foram separados em quatro, distinguindo o que é do **cliente** do que é do **usuário**.

Tabela 7: Arquivos gerados pelo servidor, no subdiretório `dados/`.

Arquivo	Conteúdo	Quando grava
<code>sessoes.log</code>	LOGIN <ip> <data e hora> e LOGOUT <ip> <data e hora>	A cada entrada e saída de cliente
<code>servidor.log</code>	Os mesmos eventos que aparecem na tela do servidor	A cada evento de conexão
<code>fila.txt</code>	Retrato atual da fila completa	Reescrito a cada ADD
<code>historico.txt</code>	Registro permanente: <data e hora> ADD <id> <nome>	A cada ADD

O arquivo `historico.txt` atende a dois dos itens pedidos pelo enunciado ao mesmo tempo. Cada linha registra um usuário cadastrado com data e hora, servindo tanto como cadastro de usuários quanto como histórico.

4 Retransmissão de mensagens

4.1 O que precisava ser garantido

O TCP garante entrega ordenada e sem perda enquanto a conexão está de pé. Ele garante que os bytes chegaram à outra máquina, mas não que a aplicação do outro lado processou o pedido. Se o servidor estiver vivo e travado, ou se a conexão cair no meio de uma operação, o TCP não resolve o problema do cliente.

A retransmissão implementada é, portanto, do nível da aplicação. Ela responde a duas perguntas: como o cliente sabe que seu comando foi processado, e o que ele faz quando não sabe.

4.2 Mecanismo adotado

O mecanismo tem três partes, e nenhuma delas usa uma mensagem de confirmação separada.

1. A resposta é a confirmação. Toda resposta listada na Seção 2.3 funciona como confirmação do pedido correspondente. O `ADD_OK` confirma que o `ADD` foi recebido e processado. Não existe uma mensagem `ACK` à parte.

2. Tempo limite e reenvio. Depois de enviar um comando, o cliente espera a resposta por até **3 segundos**. Se o prazo estourar, ele reenvia o mesmo comando, até um total de **3 tentativas**. Se as três falharem, avisa o operador com `Servidor não respondeu`, tente novamente. e volta ao menu.

3. Número de sequência e idempotência. Cada comando carrega um número de sequência próprio daquela conexão. O servidor guarda o último número processado. Se receber um número repetido, ele devolve a mesma resposta de antes **sem processar de novo**. É isso que impede um reenvio de cadastrar o mesmo usuário duas vezes.

Do lado do cliente há uma regra complementar. Uma resposta só é exibida se houver um comando esperando por ela. Cópias atrasadas, que chegam depois de o cliente ter desistido, são descartadas em silêncio. Sem essa regra, uma cópia atrasada seria confundida com a resposta do comando seguinte.

4.3 Avaliação honesta do mecanismo

Sobre uma única conexão TCP, o reenvio é em boa parte redundante, porque o próprio TCP já retransmite segmentos perdidos. Vale reconhecer isso.

O que de fato agrega é o **número de sequência**. Ele é o que torna a operação idempotente e impede que um comando repetido produza efeito duplicado. Essa garantia o TCP não oferece, porque ela é sobre o processamento na aplicação, não sobre a entrega dos bytes.

5 Testes e análise

5.1 Como os testes foram feitos

Os testes seguiram três frentes, todas repetidas a cada alteração do código:

1. **Teste funcional** — dois clientes reproduzindo a sequência das telas do enunciado.
2. **Casos especiais** — um programa conversando direto com o servidor pelo socket, enviando comandos válidos e inválidos e conferindo cada resposta. Falar o protocolo em texto permite testar situações que o `./cliente` não deixaria acontecer, como um comando malformatado ou um comando antes do login.

3. **Carga** — o programa `carga.c`, com 100, 1.000 e 10.000 clientes.

5.2 Teste funcional

Tabela 8: Teste funcional com dois clientes conectados ao mesmo servidor.

#	Ação	Resultado esperado	Obtido
1	Clientes A e B conectam	Conectado ao servidor. e Servidor: LOGIN_OK	OK
2	A cadastra 12/Marcos Castro e 13/Juarez Soares Pedro	Usuário Marcos Castro adicionado.	OK
3	B recebe as notificações	[Broadcast] Novo usuário: com os dois nomes	OK
4	B cadastra 21/Jonas Baleia e 11/Pedro Casagrande	Confirmação em B, <i>broadcast</i> em A	OK
5	A escolhe 2	Os quatro usuários, entre cabeçalho e rodapé	OK
6	A escolhe 3	Só 21 - Jonas Baleia e 11 - Pedro Casagrande, cadastrados por B	OK
7	A escolhe 3 de novo	Heartbeat: ALIVE, pois nada mudou desde a verificação anterior	OK
8	Ambos escolhem 0	Cliente desconectado. na tela do servidor	OK

O passo 6 é onde seguimos o texto da Observação em vez do *print* do enunciado. O *print* mostra a fila inteira, e o sistema devolve apenas os usuários cadastrados por outros clientes. O passo 7 mostra o outro lado da mesma regra: repetida sem novidade, a operação devolve ALIVE, exatamente como no *print* do enunciado.

Esta execução é a mesma que aparece nas capturas da Seção 7. Os quatro cadastros ficaram registrados em `dados/historico.txt`, com data e hora, e podem ser conferidos na Figura 12.

5.3 Casos especiais: 30 de 30 aprovados

Foram testados 30 casos especiais, agrupados por tema. Todos passaram. Em todos os casos de comando inválido, **a conexão permanece aberta** — um erro do operador não derruba o cliente.

Tabela 9: Casos especiais testados e resposta obtida.

#	Situação testada	Resposta obtida
<i>Autenticação</i>		
1	Senha errada	LOGIN_FAIL
2	Conexão após falha de login	Encerrada pelo servidor
3	LOGIN sem a senha	LOGIN_FAIL
4	Comando antes do login	LOGIN_FAIL
5	Credencial correta	LOGIN_OK
<i>Comandos malformados</i>		

#	Situação testada	Resposta obtida
6	Comando inexistente	ERRO Comando desconhecido
7	ADD sem argumento	ERRO Formato esperado...
8	ADD só com a sequência	ERRO Formato esperado...
9	ADD com identificador não numérico	ERRO Formato esperado...
10	ADD sem o nome	ERRO Nome invalido...
11	LIST sem número de sequência	ERRO Formato esperado...
12	HEARTBEAT sem número de sequência	ERRO Formato esperado...
13	Linha em branco	Ignorada; a conexão segue
<i>Validação de dados</i>		
14	Identificador negativo	ERRO 0 identificador deve ser um numero positivo
15	Identificador zero	ERRO ...numero positivo
16	Nome com 60 caracteres	ERRO Nome invalido, vazio ou longo demais
17	Nome com espaços	Aceito
18	Nome com acentos	Aceito
<i>Retransmissão, lado do servidor</i>		
19	ADD repetido com o mesmo número de sequência	Mesma resposta ADD_OK 10 Abel
20	Fila após o ADD repetido	Um único registro; não duplicou
21	Sequência menor que a última processada	ERRO Numero de sequencia invalido
22	Sequência zero	ERRO Numero de sequencia invalido
<i>Heartbeat e broadcast</i>		
23	<i>Broadcast</i> chega ao outro cliente	[Broadcast] Novo usuario: Breno
24	<i>Heartbeat</i> com novidade	Só 30 - Breno, cadastrado por outro
25	O mesmo <i>heartbeat</i> reenviado	Bloco idêntico ao anterior
26	<i>Heartbeat</i> seguinte, sem novidade	ALIVE
27	Esse <i>heartbeat</i> reenviado	ALIVE de novo
28	Novo cadastro e novo <i>heartbeat</i>	Só 40 - Dario, o registro novo
29	Esse <i>heartbeat</i> reenviado	Mesmo bloco
<i>Falha de conexão</i>		
30	Cliente cai sem enviar SAIR	O servidor detecta, remove a sessão e continua atendendo os demais

Análise. Os casos 25, 27 e 29 são os mais importantes do conjunto. Eles comprovam a idempotência: reenviar um comando devolve exatamente a mesma resposta e não avança o marcador do que já foi visto. Sem essa propriedade, o mecanismo de retransmissão da Seção 4 seria inseguro, porque cada reenvio produziria um efeito novo.

O caso 30 comprova que a queda abrupta de um cliente é tratada como um evento normal. O `recv()` devolve zero quando o outro lado fecha, e a thread sai do laço, remove a sessão e fecha o socket, sem afetar os demais.

Três testes dirigidos à delimitação e à serialização. As duas peças mais delicadas do protocolo são a separação das mensagens e a garantia de que um bloco de várias linhas não é partido. Elas foram testadas em separado, por um programa que fala direto com o socket. Enviadas duas mensagens completas em um único `write`, o servidor processou as duas, na ordem certa. Enviada uma mensagem partida em dois `write`, com pausa entre eles, o

servidor remontou a linha corretamente. E com 25 listagens longas pedidas enquanto outro cliente cadastrava sem parar, nenhum bloco saiu partido: todos os *broadcast* que chegaram durante as listagens apareceram fora dos blocos.

5.4 Retransmissão vista do lado do cliente

O comportamento do cliente foi testado contra um servidor de mentira, que aceita o login e depois não responde a mais nada. O objetivo era forçar o tempo limite.

```
t = 0,0s   servidor recebeu: ADD 1 10 Abel
t = 3,0s   servidor recebeu: ADD 1 10 Abel
t = 6,0s   servidor recebeu: ADD 1 10 Abel
```

Figura 2: Registro do servidor de mentira durante o teste de retransmissão.

Os resultados confirmaram o projetado:

- exatamente 3 envios, um a cada 3 segundos;
- os três com o mesmo número de sequência, que é o que permite ao servidor reconhecer o reenvio;
- o cliente avisou o operador ao fim das tentativas;
- o servidor de mentira enviou depois 3 cópias atrasadas da resposta, e o cliente ignorou todas, porque já não havia comando esperando;
- o comando seguinte funcionou normalmente, sem confusão.

5.5 Compilação

Tabela 10: Verificações de compilação.

Verificação	Resultado
make sem parâmetros	Gera exatamente cliente e servidor
Avisos com -Wall -Wextra	Nenhum
Avisos com -Wpedantic -Wshadow -Wconversion -Wsign-conversion -Wcast-align -Wnull-dereference	Nenhum
Compilação com -std=c99, -std=c11 e -std=c17, com e sem -O2	Sem erro e sem aviso nas oito combinações

6 Teste de carga com 100, 1.000 e 10.000 clientes

6.1 Como a carga foi gerada

A geração é automática, feita pelo programa `carga.c` — o segundo código cliente que o enunciado permite expressamente. Ele repete `socket()`, `connect()` e `login` N vezes, e recebe a quantidade como parâmetro:

```
make carga
./carga 100
./carga 1000
./carga 10000
```

Código 3: Comandos usados no teste de carga.

Duas providências foram necessárias para o teste medir o que devia medir. A primeira: as conexões ficam **todas abertas ao mesmo tempo** até o operador pressionar ENTER. Sem isso, o programa estaria abrindo e fechando conexões em sequência, e o servidor nunca precisaria sustentar mais de uma por vez. A segunda: cada cliente gerado **também se autentica**, ocupando uma sessão no servidor exatamente como um `./cliente`.

O servidor foi reiniciado antes de cada teste, para a numeração das conexões começar em [#1]. Antes do teste de 10.000, o limite de descritores de arquivo do terminal foi elevado com `ulimit -n 20000`.

6.2 Resultados

Tabela 11: Resultado dos três testes de carga.

Clientes	Conexões abertas	Falhas	Numeração no servidor	Sessões autenticadas
100	100	0	[#1] a [#100]	100
1.000	1.000	0	[#1] a [#1000]	1.000
10.000	10.000	0	[#1] a [#10000]	10.000

Depois de cada teste, um cliente comum foi executado e operou normalmente. Isso mostra que o servidor não ficou em estado ruim após sustentar a carga.

As capturas abaixo mostram a tela do servidor no começo e no fim de cada teste. O par comprova a numeração contínua: a primeira conexão sai como [#1] e a última como [#N], com a origem de cada uma. É assim que se identificam todas as conexões sem precisar de uma captura por linha.

```
o victor@duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ ./servidor
Servidor iniciado na porta 8080
Novo cliente conectado. [#1] 127.0.0.1:39380
Novo cliente conectado. [#2] 127.0.0.1:39384
Novo cliente conectado. [#3] 127.0.0.1:39386
Novo cliente conectado. [#4] 127.0.0.1:39402
Novo cliente conectado. [#5] 127.0.0.1:39406
Novo cliente conectado. [#6] 127.0.0.1:39416
Novo cliente conectado. [#7] 127.0.0.1:39418
Novo cliente conectado. [#8] 127.0.0.1:39430
Novo cliente conectado. [#9] 127.0.0.1:39448
Novo cliente conectado. [#10] 127.0.0.1:39442
Novo cliente conectado. [#11] 127.0.0.1:39458
Novo cliente conectado. [#12] 127.0.0.1:39474
Novo cliente conectado. [#13] 127.0.0.1:39482
Novo cliente conectado. [#14] 127.0.0.1:39498
```

(a) Início: de [#1] em diante

```
Novo cliente conectado. [#88] 127.0.0.1:40180
Novo cliente conectado. [#89] 127.0.0.1:40192
Novo cliente conectado. [#90] 127.0.0.1:40200
Novo cliente conectado. [#91] 127.0.0.1:40210
Novo cliente conectado. [#92] 127.0.0.1:40212
Novo cliente conectado. [#93] 127.0.0.1:40222
Novo cliente conectado. [#94] 127.0.0.1:40234
Novo cliente conectado. [#95] 127.0.0.1:40246
Novo cliente conectado. [#96] 127.0.0.1:40256
Novo cliente conectado. [#97] 127.0.0.1:40268
Novo cliente conectado. [#98] 127.0.0.1:40280
Novo cliente conectado. [#99] 127.0.0.1:40290
Novo cliente conectado. [#100] 127.0.0.1:40298
```

(b) Fim: até [#100]

Figura 3: Teste com 100 clientes, visto do servidor.


```

victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ ./servidor
Servidor iniciado na porta 8080
Novo cliente conectado. [#1] 127.0.0.1:57262
Novo cliente conectado. [#2] 127.0.0.1:57266
Novo cliente conectado. [#3] 127.0.0.1:57280
Novo cliente conectado. [#4] 127.0.0.1:57292
Novo cliente conectado. [#5] 127.0.0.1:57302
Novo cliente conectado. [#6] 127.0.0.1:57316
Novo cliente conectado. [#7] 127.0.0.1:57338
Novo cliente conectado. [#8] 127.0.0.1:57336

```

(a) Início: de [#1] em diante

```

Novo cliente conectado. [#992] 127.0.0.1:37706
Novo cliente conectado. [#993] 127.0.0.1:37716
Novo cliente conectado. [#994] 127.0.0.1:37732
Novo cliente conectado. [#995] 127.0.0.1:37746
Novo cliente conectado. [#996] 127.0.0.1:37760
Novo cliente conectado. [#997] 127.0.0.1:37764
Novo cliente conectado. [#998] 127.0.0.1:37772
Novo cliente conectado. [#999] 127.0.0.1:37782
Novo cliente conectado. [#1000] 127.0.0.1:37792

```

(b) Fim: até [#1000]

Figura 4: Teste com 1.000 clientes, visto do servidor.

```

Novo cliente conectado. [#9991] 127.0.0.1:46600
Novo cliente conectado. [#9992] 127.0.0.1:46604
Novo cliente conectado. [#9993] 127.0.0.1:46614
Novo cliente conectado. [#9994] 127.0.0.1:46630
Novo cliente conectado. [#9995] 127.0.0.1:46640
Novo cliente conectado. [#9996] 127.0.0.1:46652
Novo cliente conectado. [#9997] 127.0.0.1:46660
Novo cliente conectado. [#9998] 127.0.0.1:46664
Novo cliente conectado. [#9999] 127.0.0.1:46680
Novo cliente conectado. [#10000] 127.0.0.1:46696

```

(a) Servidor: até [#10000]

```

Conexoes estabelecidas: 9800 de 9800
Conexoes estabelecidas: 9900 de 9900
Conexoes estabelecidas: 10000 de 10000

Resultado: 10000 conexoes abertas, 0 falhas.
As 10000 conexoes permanecem ativas simultaneamente.
Pressione ENTER para encerra-las...

```

(b) Gerador: 10.000 de 10.000, 0 falhas

Figura 5: Teste com 10.000 clientes. À esquerda, a última conexão registrada pelo servidor; à direita, o fecho do gerador, com as 10.000 conexões abertas ao mesmo tempo e nenhuma falha.

```

victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ ./carga 1000
Abrindo 1000 conexoes em 127.0.0.1:8080

Conexoes estabelecidas: 100 de 100
Conexoes estabelecidas: 200 de 200
Conexoes estabelecidas: 300 de 300
Conexoes estabelecidas: 400 de 400
Conexoes estabelecidas: 500 de 500
Conexoes estabelecidas: 600 de 600
Conexoes estabelecidas: 700 de 700
Conexoes estabelecidas: 800 de 800
Conexoes estabelecidas: 900 de 900
Conexoes estabelecidas: 1000 de 1000

Resultado: 1000 conexoes abertas, 0 falhas.
As 1000 conexoes permanecem ativas simultaneamente.
Pressione ENTER para encerra-las...

Todas as conexoes foram encerradas.
victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$

```

Figura 6: Tela do gerador ./carga 1000. Ele informa o progresso a cada 100 conexões, o total aberto e o número de falhas, e mantém tudo ativo até o operador pressionar ENTER.

6.3 Análise dos resultados

Por que 10.000 threads couberam. Cada thread é criada com pilha de 256 KB, definida por `pthread_attr_setstacksize`. Com o padrão do sistema, de 8 MB, 10.000 threads reservariam 80 GB de espaço de endereçamento e o teste seria impossível. Com 256 KB, o total fica em torno de 2,5 GB de memória virtual, que é espaço reservado e não ocupado fisicamente.

Por que as linhas não saem em ordem numérica perfeita. No teste de 100 clientes, a linha [#99] apareceu depois da [#100]. Isso não é defeito. Cada conexão é atendida por

uma thread diferente, e a ordem em que elas escrevem no terminal depende de qual thread o sistema escalona primeiro. O número é atribuído no momento da aceitação, então a contagem está correta. Na prática, é uma evidência visual de que o servidor é mesmo concorrente.

7 Telas do funcionamento correto

Esta seção mostra o sistema em operação, na mesma sequência do teste funcional descrito na Seção 5.2. Todas as capturas vêm da execução registrada em dados/historico.txt, reproduzida na Figura 12.

```
o victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ ./servidor
Servidor iniciado na porta 8080
Novo cliente conectado. [#1] 127.0.0.1:50326
Novo cliente conectado. [#2] 127.0.0.1:48738
Cliente desconectado. [#2] 127.0.0.1:48738
Cliente desconectado. [#1] 127.0.0.1:50326
█
```

Figura 7: Servidor em execução. A primeira linha é Servidor iniciado na porta 8080; em seguida aparecem os dois clientes conectando e desconectando, cada um com seu número de conexão e sua origem.

```
o victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ ./cliente
Conectado ao servidor.
Servidor: LOGIN_OK

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
1
ID: 12
Nome: Marcos Castro
Usuário Marcos Castro adicionado.

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
1
ID: 12
Nome: Joarez Soares Pedro
Usuário Joarez Soares Pedro adicionado.

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
█
```

Figura 8: Cliente A. A captura mostra a sessão inteira: Conectado ao servidor., a resposta LOGIN_OK, o menu de quatro opções e dois cadastros feitos em sequência, cada um confirmado pelo servidor.

```

o victor@duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ ./cliente
Conectado ao servidor.
Servidor: LOGIN OK

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
[Broadcast] Novo usuário: Marcos Castro
[Broadcast] Novo usuário: Juarez Soares Pedro

```

(a) Cliente B recebe os cadastros feitos em A

```

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
[Broadcast] Novo usuário: Jonas Baleia
[Broadcast] Novo usuário: Pedro Casagrande
2

===== FILA =====
12 - Marcos Castro
13 - Juarez Soares Pedro
21 - Jonas Baleia
11 - Pedro Casagrande
=====

```

(b) Cliente A recebe os cadastros feitos em B e pede a fila

Figura 9: Notificação em tempo real, vista dos dois lados. Nenhum dos dois clientes pediu essas linhas: elas chegaram sozinhas, assim que o outro cadastrou. À direita, a opção 2 logo depois, com os quatro usuários entre o cabeçalho e o rodapé.

```

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
3
Heartbeat:
===== FILA =====
21 - Jonas Baleia
11 - Pedro Casagrande
=====

```

(a) Heartbeat com novidade

```

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
3
Heartbeat: ALIVE

```

(b) Heartbeat sem novidade

Figura 10: Comportamento do *heartbeat* no cliente A, conforme a Observação do enunciado. À esquerda, devolve apenas 21 - Jonas Baleia e 11 - Pedro Casagrande, cadastrados pelo cliente B. À direita, a mesma opção repetida em seguida devolve ALIVE, porque nada mudou desde a verificação anterior.

```

1 - Adicionar usuário
2 - Ver fila
3 - Heartbeat
0 - Sair
1
ID: -7
ID inválido. Informe um número inteiro positivo.

```

Figura 11: Validação no cliente. O identificador -7 é recusado antes mesmo de virar mensagem de rede, e o operador volta ao menu com a conexão intacta. O servidor faz a mesma checagem por conta própria, como mostram os casos 14 e 15 da Tabela 9.

```
● victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ cat dados/fila.txt
===== FILA =====
12 - Marcos Castro
13 - Juarez Soares Pedro
21 - Jonas Baleia
11 - Pedro Casagrande
=====
● victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$ cat dados/historico.txt
2026-07-28 21:35:30 ADD 12 Marcos Castro
2026-07-28 21:35:56 ADD 13 Juarez Soares Pedro
2026-07-28 21:38:03 ADD 21 Jonas Baleia
2026-07-28 21:38:36 ADD 11 Pedro Casagrande
○ victor@Duckmain:~/ClienteServidor/ClienteServidor/ClienteServidor_Victor$
```

Figura 12: Persistência em arquivo. O `fila.txt` guarda o retrato atual da fila, no mesmo formato da resposta de LIST; o `historico.txt` guarda o registro permanente de cada inserção, com data e hora.

8 Conclusão

O trabalho atingiu o objetivo proposto. O sistema funciona corretamente e segue as orientações e as exigências do enunciado: cliente e servidor são iniciados sem parâmetros, o servidor atende vários clientes ao mesmo tempo, a fila é atualizada em tempo real e os dados ficam gravados em arquivo. Todos os testes previstos foram executados com sucesso, inclusive o de carga com 10.000 clientes.

A parte mais difícil do desenvolvimento foi delimitar as mensagens. Os códigos vistos em aula trocam uma única mensagem por conexão, então nenhum deles precisa resolver esse problema. Aqui a conexão fica aberta e carrega dezenas de mensagens, e o TCP entrega os bytes sem separar uma da outra. Foi preciso criar a convenção de uma linha por mensagem e uma leitura que vai juntando os bytes recebidos até a linha ficar completa.

Escolher a técnica de múltiplos clientes também deu trabalho. O enunciado permite três caminhos: multiplexação, `fork()` e `threads`. Deixei a multiplexação de lado por ser a técnica com que tive menos contato: implementá-la e depois saber explicá-la com segurança levaria mais tempo do que eu tinha. O `fork()` parecia o caminho natural, mas processos têm memória separada, e compartilhar a fila entre eles exigiria comunicação entre processos. As `threads` compartilham a memória sozinhas, e foi por isso que as escolhi. Em troca, foi preciso proteger a fila com um *mutex*.

O teste de 10.000 clientes não funcionou na primeira tentativa. O programa parava por falta de memória, e demorei para entender o motivo: cada thread reserva 8 MB de pilha por padrão, o que daria 80 GB só de espaço reservado. Reduzir a pilha para 256 KB resolveu, e o teste passou sem nenhuma falha.

Ao longo do caminho surgiram várias ideias que eu gostaria de ter colocado em prática, mas o prazo não deixou. A que mais me deixou curioso foi a multiplexação: seria muito interessante construir o mesmo sistema das duas maneiras e comparar quanto cada uma custa por cliente. Também pensei em ler o endereço do servidor de um arquivo de configuração, o que evitaria recompilar a cada troca de máquina, e em substituir o vetor fixo por uma estrutura que cresce sozinha, tirando o limite de 20.000 registros. Fica a vontade de voltar ao código depois da entrega e experimentar cada uma delas.

Referências bibliográficas

- [1] FREE SOFTWARE FOUNDATION. *GNU Make Manual*. Disponível em: <http://www.gnu.org/software/make/manual/make.html>. Acesso em: 28 jul. 2026.
- [2] COLBY COLLEGE. *Makefile Tutorial*. Disponível em: <http://www.cs.colby.edu/maxwell/courses/tutorials/maketutor/>. Acesso em: 28 jul. 2026.
- [3] THE LINUX MAN-PAGES PROJECT. *Linux Programmer's Manual*. Páginas consultadas: `socket(2)`, `bind(2)`, `listen(2)`, `accept(2)`, `send(2)`, `recv(2)`, `setsockopt(2)`, `pthread_create(3)`, `pthread_mutex_init(3)`, `pthread_cond_timedwait(3)`, `pthread_attr_setstacksize(3)`, `sscanf(3)`. Disponível em: <https://man7.org/linux/man-pages/>. Acesso em: 28 jul. 2026.
- [4] BARBOSA FILHO, Rubens. *Códigos de exemplo da disciplina de Redes de Computadores*: `bind.c`, `escrevendo.c`, `fork.c`, `multithread.c`, `porta.c`, `redes.c`. Dourados: Universidade Estadual de Mato Grosso do Sul, 2026.